Structured Safety Judge Alignment of Llama-3.2-3B via QLoRA: End-to-End Pipeline, Reproducibility, and Empirical Results

Shyam Karthick
GitHub Repository: https://github.com/ShyamKarthickSec/JudgeLLM-Finetuning
LoRA Adapter Output: `outputs/llama32_3b_judge_qlora/adapter_final`

***Abstract*—This document reports an end-to-end experiment to align an instruction-tuned language model into a structured Safety Judge suitable for gating actions in embodied-agent and robotics-like instruction pipelines. We fine-tune `meta-llama/Llama-3.2-3B-Instruct` using QLoRA (4-bit quantization with trainable LoRA adapters) on a curated 1K-example dataset where each sample contains a conversational prompt and a gold JSON decision. The target behavior is generation of a parser-valid JSON object containing a verdict label (`ALLOW`, `DENY`, `ASK_CLARIFY`), a confidence value, and rule-grounded violations. The full training and evaluation pipeline is implemented using an open-source stack (Transformers, TRL, PEFT, bitsandbytes) and executed under consumer GPU constraints. We evaluate the base model versus the fine-tuned adapter on a held-out test split using metrics aligned with operational safety requirements: parser-valid JSON object extraction rate, verdict correctness, rule grounding accuracy, unsafe-case recall, and unsafe false negatives. Across 100 held-out test examples, the fine-tuned model improves JSON object extraction rate from 0.94 to 1.00, verdict accuracy from 0.61 to 0.90, rule exact match from 0.34 to 0.78, and increases unsafe-case detection (DENY recall) from 0.775 to 0.95 while reducing unsafe false negatives from 0.225 to 0.05. These results validate the training and evaluation pipeline and motivate subsequent robustness testing under adversarial prompt manipulation and distribution shift.**

I. Introduction

As LLM-based agents increasingly interact with the physical world, the ability to filter unsafe or policy-violating instructions becomes a core requirement for reliable deployment. In embodied settings, systems are exposed not only to benign commands but also to hazardous instructions, privacy-invasive requests, ambiguous tasks requiring clarification, and adversarially crafted prompts intended to bypass safety constraints. A safety judge module in such pipelines must satisfy constraints that extend beyond general language generation: it must produce outputs in a deterministic, machine-parseable format; it must provide decisions that are correct under a specified policy; and it must support auditing by grounding decisions in explicit rule identifiers and evidence.

This report documents a complete experiment that converts an instruction-tuned model into a structured safety judge via supervised fine-tuning. The experiment is designed to validate the pipeline end-to-end—data preparation, prompt formatting, QLoRA training, checkpointing, adapter export, and evaluation—before scaling to larger models and more adversarial evaluation regimes.

II. Problem Statement

A base instruction-tuned model is not inherently specialized for safety judgment tasks under structured schema constraints. In practice, it may generate text outside the required JSON schema, provide plausible but policy-incorrect verdicts, omit rule grounding, or fail safety-critical cases by allowing instructions that should be denied. These failure modes are particularly problematic in embodied pipelines where non-parseable outputs break automatic enforcement and unsafe false negatives directly correspond to hazardous executions.

The objective of this experiment is to align the model to a narrow, operationally motivated behavior: given a conversational context describing an instruction to an embodied agent, the model must output a parser-extractable JSON object containing (i) a categorical verdict (`ALLOW`, `DENY`, `ASK_CLARIFY`), (ii) a confidence score, and (iii) policy-grounded violations with rule identifiers and concise evidence. We treat reduction of unsafe false negatives (gold `DENY` predicted as non-`DENY`) as the primary safety objective, while maintaining structured output compliance and improving interpretability via correct rule grounding.

III. Methodology

A. Base Model and Fine-Tuning Strategy

We use `meta-llama/Llama-3.2-3B-Instruct` as the base model due to its strong instruction-following prior and suitability for iterative experimentation. To enable fine-tuning under limited GPU memory, we employ QLoRA. In this configuration, the frozen base model is loaded in 4-bit quantized form (NF4 quantization), and only lightweight adapter parameters are updated during training. This significantly reduces the memory footprint and enables training under consumer GPU constraints without modifying the full base weights. The adapter is trained on standard Llama projection modules (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) with adapter hyperparameters `r=16`, `lora_alpha=32`, and `lora_dropout=0.05`. The final adapter is exported to `outputs/llama32_3b_judge_qlora/adapter_final` for inference-time composition with the base model.

B. Dataset and Target Output Schema

Training uses a curated dataset of 1,000 examples split into 800 training, 100 validation, and 100 held-out test samples. Each example is represented as a chat-style `messages[]` sequence and includes gold labels `expected_verdict` and `expected_rules`. The gold assistant completion is a JSON object constrained to include a `verdict`, `confidence`, `violations` (with `rule_id` and evidence), and `notes`. This schema is operationally motivated: downstream systems can parse the output deterministically, enforce the verdict programmatically, and audit decisions by inspecting rule identifiers.

C. Prompt Construction and Completion-Only Supervision

Each dataset row contains a multi-turn chat. The final assistant message is treated as the supervised target completion. During preprocessing, each example is converted into (i) a `prompt` constructed from all messages prior to the final assistant turn and (ii) a `completion` equal to the final assistant JSON plus an end-of-sequence token. The prompt is rendered using the model's native chat template with the generation prompt enabled. Training uses completion-only loss so that gradients are applied to generated completion tokens, aligning the model to the structured decision output while keeping the conditioning context fixed.

D. Training Configuration and Execution Environment

Training is executed locally under WSL2 with an NVIDIA GeForce RTX 4070 Laptop GPU (12GB VRAM). The environment uses Python 3.11.14, PyTorch 2.6.0+cu124, CUDA 12.4, Transformers 5.2.0, TRL 0.28, PEFT, and bitsandbytes. The fine-tuning run uses a maximum sequence length of 512 tokens. We train for two epochs with per-device batch size 1 and gradient accumulation of 15. A cosine learning-rate schedule is used with learning rate 2×10^{-4} and a warmup fraction of 0.05. Gradient checkpointing is enabled to reduce memory overhead. Mixed precision (bf16 or fp16) is selected based on GPU capability. Checkpoints are saved and evaluated at 100-step intervals with a total checkpoint limit of two.

E. Evaluation Design: Base vs. Adapter

We evaluate both the base model and the fine-tuned adapter model on the held-out test split under identical inference conditions. For each test example, the prompt is constructed by removing the gold assistant completion and rendering the remaining messages using the same chat template. The model generates a completion deterministically (no sampling). The output is parsed using a recoverable JSON extraction heuristic that selects the first parseable JSON object from the generation. JSON validity is therefore defined as the proportion of outputs from which a JSON object can be successfully extracted and parsed; this metric does not enforce strict schema-only compliance and may include cases where additional surrounding text is present. Predicted verdicts and rule identifiers are extracted from the parsed object. The evaluation pipeline produces aggregate metrics and per-sample artifacts (raw generations, parsed outputs, and comparison tables) for audit and error analysis.

All scripts and artifacts required to reproduce training and evaluation are provided in the public repository. Reproduction requires the documented software environment and access prerequisites for the base model (e.g., Hugging Face authentication and license acceptance), as well as a compatible CUDA-enabled GPU configuration.

IV. Results

Table I reports base model performance versus the fine-tuned adapter on the 100-sample held-out test set.

TABLE I
BASE VS. FINE-TUNED ADAPTER RESULTS ON HELD-OUT TEST SET (N=100)

Metric	Base	Adapter	Delta
JSON object extraction rate	0.9400	1.0000	+0.0600
Verdict accuracy	0.6100	0.9000	+0.2900
Rules exact match	0.3400	0.7800	+0.4400
DENY recall	0.7750	0.9500	+0.1750
Unsafe false negative rate (DENY)	0.2250	0.0500	-0.1750

V. Discussion

The experimental results demonstrate that completion-only supervised fine-tuning under QLoRA can strongly align a small instruction model toward structured safety judgment behavior. The improvement in parser-valid JSON object extraction rate indicates greater structural reliability of outputs. Verdict accuracy and rule grounding both improve substantially, suggesting stronger alignment to policy-consistent decision making rather than surface-level token prediction. Most importantly, unsafe false negatives are significantly reduced, indicating that the aligned model more reliably blocks unsafe instructions. These results validate the pipeline and support further experimentation under adversarial and out-of-distribution conditions.

VI. Limitations

Evaluation is conducted primarily in-distribution and does not yet measure robustness under adversarial prompt manipulation or significant distribution shift. JSON validity is measured using a recoverable extraction heuristic rather than strict schema enforcement. The held-out test set contains 100 samples; larger-scale evaluation would provide tighter statistical confidence.

VII. Conclusion

This report presents a reproducible pipeline for aligning `Llama-3.2-3B-Instruct` into a structured safety judge using QLoRA under consumer GPU constraints. The fine-tuned adapter demonstrates substantial improvements in structured output reliability, verdict correctness, rule grounding, and safety-critical detection of unsafe instructions. These results establish a defensible baseline for scaling experiments and extending evaluation to adversarial and out-of-distribution safety scenarios.